Global Project Assignment

ECE 466 Homework 6

Question 1a Sentiment classification

The course notebook pools the 50,000 labeled IMDB reviews and creates a new stratified 90/10 split: 45,000 training reviews and 5,000 test reviews, with equal positive and negative counts. Seed 466 keeps this split fixed for all models. The test results below refer to this course split.

Reviews are lowercased;
 tags and punctuation are removed. The Keras tokenizer is fitted only on training reviews, keeps token indices below 10,000, and maps other words to <OOV>. Long reviews keep their last 80 tokens; short reviews receive leading zeros. This original prepadding is retained without sequence packing.

For a batch of B reviews, token IDs have shape B × 80, embeddings have shape B × 80 × 10, and the embedding table has 158,904 × 10 entries, following total_word in the notebook. The hidden size H is 164. Each model has one recurrent layer and zero recurrent dropout. Hidden states, and the LSTM cell state, start at zero for each review.

Architecture 1 uses a simple RNN with tanh:

$$h_t = tanh(W_x e_t + W_h h_{t-1} + b).$$

Architecture 2 uses an LSTM. Concatenate the token embedding and previous hidden state, then compute its gates and states:

$$a_t = [e_t;\ h_{t-1}].$$

$$i_t = \sigma(W_i a_t + b_i),\quad f_t = \sigma(W_f a_t + b_f).$$

$$g_t = tanh(W_g a_t + b_g),\quad o_t = \sigma(W_o a_t + b_o).$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t,\quad h_t = o_t \odot tanh(c_t).$$

Here σ is the sigmoid function and ⊙ is elementwise multiplication. The classifier uses the final hidden state, not the LSTM cell state. For a bidirectional model, the final forward and reverse states are concatenated; using the last output timestep would miss the final reverse state.

$$s = h_n[-1]\ (one\ direction),\quad s = concat(h_n[-2], h_n[-1])\ (two\ directions).$$

$$z = W_{out} s + b_{out},\quad predicted\ class = argmax(z).$$

The linear input width is 164 or 328, and the output has two logits. All four models use CrossEntropyLoss, Adam with learning rate 0.0001, batch size 64, and 20 epochs. Class 0 is negative and class 1 is positive. Test evaluation disables gradients and uses evaluation mode.

$$Test\ accuracy\ (\%) = 100 \times \frac{correct\ test\ predictions}{5000}.$$

Model	Bidirectional	Linear input	Correct / Total	Accuracy
RNN	False	164	3,873 / 5,000	77.46%
RNN	True	328	3,907 / 5,000	78.14%
LSTM	False	164	3,985 / 5,000	79.70%
LSTM	True	328	3,970 / 5,000	79.40%

Question 1b Performance comparison

Relative to the RNN, LSTM test accuracy increased by 2.24 percentage points for one direction (112 more correct reviews) and increased by 1.26 percentage points for two directions (63 more correct reviews).

Adding bidirectionality changed RNN accuracy from 77.46% to 78.14%: it increased by 0.68 percentage points (34 more correct reviews). LSTM accuracy changed from 79.70% to 79.40%: it decreased by 0.30 percentage points (15 fewer correct reviews).

LSTM gates control retained information and can reduce the difficulty of learning dependencies across many time steps. Bidirectionality combines forward and reverse summaries and adds parameters, but it does not guarantee better test accuracy under a fixed training budget. The measured effects above are from one fixed split and seed with the same 20-epoch budget.

Bonus Embedding dimension 100

The bidirectional LSTM was retrained with embedding dimension 100 instead of 10. All remaining settings and the split were unchanged. It correctly classified 4,188 of 5,000 reviews (83.76%). Accuracy increased by 4.36 percentage points relative to the bidirectional LSTM in Question 1a (79.40%), with 218 more correct reviews.

The larger embedding adds $90 \times 158{,}904 = 14{,}301{,}360$ embedding parameters and $2 \times 4 \times 164 \times 90 = 118{,}080$ input-to-gate weights. Only embedding rows with token indices below 10,000 are accessed. The accuracy gain is consistent with the larger embeddings representing useful word features more effectively in this run, at the cost of estimating more parameters from the same data.

Question 2a Baseline image captioning

The training set contains 118,287 COCO 2017 images and 591,753 caption annotations. The provided course vocabulary of 9,438 tokens is reused; the notebook's vocabulary threshold setting is 6. Training images are resized to a shorter edge of 256, randomly cropped to 224 × 224, randomly flipped horizontally, and normalized with the ImageNet mean and standard deviation.

The baseline uses an ImageNet-pretrained ResNet50. Global average pooling produces 2048 features, which a trainable linear layer maps to 512 dimensions. Caption embeddings also have 512 dimensions. A one-layer LSTM with hidden size 512 feeds a linear layer with 9,438 output logits.

The decoder input is the image embedding followed by all caption tokens except the last. The target is the complete caption: the image predicts <start>, <start> predicts the first word, and the last word predicts <end>. This preserves the course notebook's alignment.

All three models use batch size 256, Adam with learning rate 0.001, seed 466, and 1 complete epoch. Equal-length caption batches are shuffled without replacement, so each caption is included once per epoch. This replaces the notebook's replacement sampler to satisfy the full-pass requirement. CNN parameter gradients are frozen while its training-mode running statistics are retained; the image projection and decoder are trained.

Each full epoch covers 2,348 batches and 7,871,788 target tokens.

All runs use FP32. The CNN and image storage formats are 2a: channels-last; 2b: contiguous; 2c: channels-last. Floating-point reduction order can differ across these formats.

For each batch b, S_b is the sum of negative natural log-probabilities of the target tokens and N_b is the number of those tokens. The complete-epoch metrics are:

$$L = \frac{\Sigma_b S_b}{\Sigma_b N_b}, \quad Perplexity = exp(L).$$

L is the token-weighted online loss recorded while training during the final completed epoch. It is not a separate evaluation of the final checkpoint. Checkpoints are saved after each full epoch.

Part	Encoder and decoder	Training loss	Perplexity
2a	ResNet50 and LSTM	2.698328	14.855
2b	GoogLeNet and LSTM	2.739124	15.473
2c	ResNet50 and RNN	2.744996	15.565

Question 2b GoogLeNet encoder

ResNet50 is replaced with pretrained GoogLeNet. Its classification head and auxiliary outputs are removed, and its 1024 features are projected to the same 512-dimensional embedding. The LSTM decoder, vocabulary, optimizer settings and epoch budget remain the same. Relative to the baseline, training loss increased by 0.040795 nats per token and perplexity increased by 4.16 percent. The different pretrained features and their projection change how visual information reaches the decoder. The measured difference is for this training run and budget.

Question 2c RNN decoder

The original ResNet50 encoder is retained. The decoder uses a one-layer tanh RNN with hidden size 512 instead of the LSTM, with the same embeddings, targets and vocabulary output layer. Relative to the baseline, training loss increased by 0.046668 nats per token and perplexity increased by 4.78 percent. The RNN has a simpler recurrent update and fewer parameters, but lacks the LSTM's gated cell state for retaining information across a caption.

Question 2d Captions for four test images

Four test2017 images were selected once with seed 466. Each model received the same images resized to 224 × 224 with deterministic normalization. The adapted get_prediction function uses greedy decoding and stops at <end> or the common length limit. The captions below are the saved outputs of the three trained models.

Images are reproduced unchanged from COCO test2017. Its supplied metadata does not include photographer names or original Flickr URLs; source and license links are given below.

Image 1 COCO ID 49919



Model	Generated caption
ResNet50 and LSTM	A woman standing in a kitchen with a pizza .
GoogLeNet and LSTM	A man is holding a banana in his hand .
ResNet50 and RNN	A woman sitting at a table with a pizza cutter .

COCO ID 49919 · Original image · CC BY-NC 2.0

The baseline identifies the kitchen, but its pizza is not visible. GoogLeNet invents someone holding a banana, while the RNN describes sitting at a table with a pizza cutter; both miss the visible kitchen activity.

Image 2 COCO ID 116033



Model	Generated caption
ResNet50 and LSTM	A person riding a motorcycle down a snow covered slope .
GoogLeNet and LSTM	A person on a surfboard in the water .
ResNet50 and RNN	A man riding a horse on a dirt road .

COCO ID 116033 · Original image · CC BY-NC-ND 2.0

The baseline recognizes the motorcycle, but the image shows it parked on a muddy track with a surfboard, not being ridden on snow. GoogLeNet mentions surfboard and water elements but invents a person on the board; the RNN substitutes a horse for the vehicle.

Image 3 COCO ID 273741



Model	Generated caption
ResNet50 and LSTM	A person riding a surfboard on a wave .
GoogLeNet and LSTM	A person on a surfboard riding a wave .
ResNet50 and RNN	A person riding a surfboard on a wave .

COCO ID 273741 · Original image · CC BY-NC-SA 2.0

All three recognize the person, surfboard and waves, but describe riding; the person is standing in the water holding the board. The baseline and RNN captions are identical, while GoogLeNet mainly changes the word order.

Image 4 COCO ID 177118



Model	Generated caption
ResNet50 and LSTM	A bench with a wooden bench and a bench .
GoogLeNet and LSTM	A bench sitting on a wooden bench in the grass .
ResNet50 and RNN	A wooden bench sitting in a field next to a tree .

COCO ID 177118 · Original image · CC BY 2.0

All three mention a bench; the baseline is repetitive, while GoogLeNet incorrectly places one bench on another. The RNN's description of a bench near a tree is more coherent, although all three omit the visible people.

The baseline better identifies the kitchen and motorcycle in these examples; the RNN gives the clearest description of the park scene. All three invent actions or objects and miss details, so the baseline's lower training loss does not make its test captions consistently correct.

Implementation sources: course HW6 notebooks and the image-captioning project by Benjamin Dwumah. Project source